

راهنمای توسعه هوش مصنوعی

AI DEVELOPMENT GUIDE

مرجع لایه هوش مصنوعی پروژه چت هوشمند — Smart Chat
مدل‌ها، آپستریم، استریم SSE و API سازگار OpenAI / Anthropic

این سند هر آنچه بین کاربر و مدل‌های زبانی اتفاق می‌افتد را مستند می‌کند: معماری چهارلایه از کلاینت تا آپستریم freemodells، کاتالوگ هفت مدل و قواعد نرمال‌سازی نام مدل، پارسر جهانی SSE، مرجع کامل اندپوینت‌های سازگار OpenAI و Anthropic با نمونه‌های curl و Python، مدیریت کلید، ماتریس خطاها و کتاب راهنمای توسعه برای افزودن مدل و اکستنشن جدید. مخاطب آن توسعه‌دهنده‌ها و مصرف‌کننده‌های API این پروژه‌اند.

Next.js 16 + app.js

۷ مدل ۳۰ اندپوینت v1/

۱۴ سپتامبر ۲۰۲۶ ۲۳ شهریور ۱۴۰۵

نسخه سند ۱.۰

معرفی و دامنه | Overview & Scope

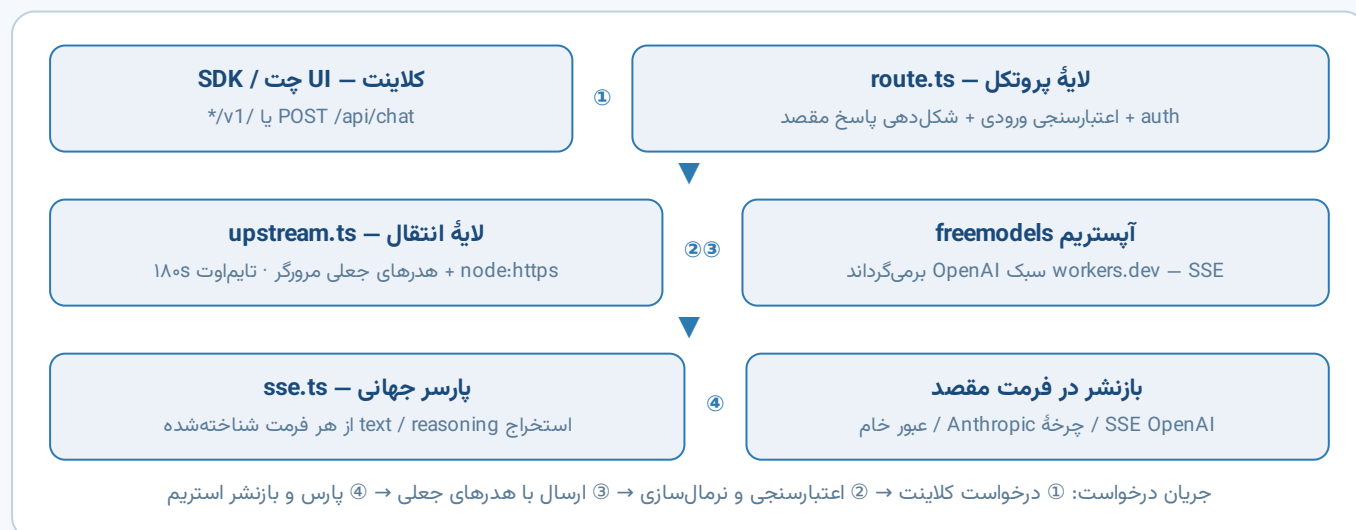
این سند مرجع عملیاتی «لایه هوش مصنوعی» پروژه است: هر آنچه بین کاربر و مدل‌های زبانی اتفاق می‌افتد. دامنه آن چهار زیرسیستم را پوشش می‌دهد: (۱) **کاتالوگ مدل‌ها** و نرمال‌سازی نام مدل، (۲) **انتقال به آپستریم** freemodls با جعل هدرهای مرورگر، (۳) **استریم SSE** و پارسر جهانی پاسخ، و (۴) **لایه سازگاری API** با دو استاندارد OpenAI و Anthropic. آنچه در دامنه نیست – نصب و اجرا، UI چت، پیکر مدل گروهی و تاریخچه تسک‌ها – در `README.md` و `HANDOFF.md` پوشش داده شده است.

راهنما از دو منظر نوشته شده است. منظر نخست «مصرف‌کننده API» است: کسی که با curl یا SDK رسمی OpenAI/Anthropic به اندپوینت‌های `/v1` وصل می‌شود و فقط قرارداد درست درخواست/پاسخ لازم دارد (بخش‌های ۵ تا ۹). منظر دوم «توسعه‌دهنده پروژه» است: کسی که می‌خواهد مدل اضافه کند، فیلد اکستنشن بسازد، رفتار استریم را عوض کند یا آپستریم را تعویض کند (بخش‌های ۲ تا ۴ و ۱۰). هر دو منظر روی یک واقعیت مشترک تکیه دارند: پروژه به سرویس رایگان freemodls متصل است و همه تصمیم‌های طراحی – از جعل هدر تا پارسر جهانی – نتیجه همین محدودیت‌اند.

قانون طلایی: هر قابلیت در دو نسخه پروژه (Next.js در `/src` و تک‌فایل `app.js`) باید همگام بماند؛ هر جا فایلی از `src/lib` معرفی می‌شود، معادل خطی آن در `app.js` هم ذکر شده است.

معماری لایه هوش مصنوعی | AI Layer Architecture

جریان یک درخواست چت از لحظه خروج از کلاینت تا بازگشت اولین توکن از چهار لایه مجزا عبور می‌کند. این جداسازی عمدی است: هر لایه مستقل تست و تعویض می‌شود و باگ هر لایه در مرزی مشخص منتسب می‌شود. کد لایه‌ها پراکنده نیست – لایه پروتکل در `route.ts`، لایه انتقال در `upstream.ts` و پارس در `sse.ts` زندگی می‌کند.



لایه انتقال (`src/lib/upstream.ts`) · معادل `openUpstream` در `app.js:2083` تنها نقطه تماس با دنیای بیرون است و سه ثابت آن قراردادهای سخت‌گیرانه‌ای هستند که نباید بی‌فکر تغییر کنند:

آدرس آپستریم

```
UPSTREAM_URL = freemodells-  
chat.freemodells.workers.dev
```

تنها اندپوینت چت سرویس؛ اگر روزی عوض شد هر دو نسخه را همزمان تغییر بدهید.

سقف بدنه درخواست

```
MAX_BODY_BYTES = 5MB → 413
```

تاریخچه‌های خیلی بلند باید قبل از ارسال کوتاه شوند؛ خطا شفاف 413 برمی‌گردد.

تایم‌اوت پاسخ

```
UPSTREAM_TIMEOUT_MS = 180_000
```

۱۸۰ ثانیه برای پوشش تفکر طولانی مدل‌ها؛ بیشتر ← 504.

هدرهای جعلی

```
Origin / Referer = freemodells.pro
```

UA کروم ۱۵۲، sec-fetch، *-sec-ch-ua* و Accept- Encoding: identity (پاسخ بدون فشرده‌سازی تا pipe مستقیم).

چرا `node:https` و نه `fetch`؟ آپستریم فقط به درخواست‌هایی که از دامنه `freemodells.pro` آمده باشند پاسخ می‌دهد و این را از هدرهای `Origin` و `Referer` تشخیص می‌دهد. مشخصات `fetch` ارسال این دو هدر را ممنوع کرده است، در حالی که `node:https` آزادانه اجازه می‌دهد. سه خط قرمز این لایه: اول، پاسخ آپستریم هرگز نباید در سرور بافر شود — تکه‌به‌تکه با `Cache-Control: no-cache, no-transform` و `X-Accel-Buffering: no` عبور کند وگرنه استریم زنده می‌میرد. دوم، قطع کلاینت باید بلافاصله آپستریم را `destroy` کند (Next از طریق `app.js`، `req.signal` از رویداد `close` سوکت) — نشی اتصال ممنوع. سوم، هدرهای `*-access-control` و `transfer-encoding` و `content-encoding` آپستریم قبل از عبور حذف می‌شوند تا پاسخ تمیز بماند.

SECTION 03

کاتالوگ مدل‌ها و انتخاب مدل | Model Catalog & Selection

۳.۱ جدول مدل‌ها

مدل‌ها در یک منبع واحد تعریف می‌شوند — `FM_MODELS` در `src/lib/models.ts` و همان ثابت در `app.js:66` — و پیکر مدل، دیتالیست و هر سه اندپوینت `v1/` از همین منبع می‌خوانند. هیچ لیست موازی‌ای در فایل دیگری وجود ندارد و نباید ساخته شود.

Model ID	نام نمایشی	سازنده	گروه پیکر	نقش پیشنهادی
claude-fable-5.1	Claude Fable 5.1	Anthropic	Claude Pro	پیش‌فرض — تعادل کیفیت برای کار عمومی
claude-sonnet-5	Claude Sonnet 5	Anthropic	Claude Pro	کدنویسی و استدلال ساخت‌یافته
claude-fable-5	Claude Fable 5	Anthropic	Claude Pro	نگارش خلاق و روایت
gpt-5.6-sol	GPT 5.6 Sol	OpenAI	ChatGPT Pro	پاسخ سریع و محاوره‌ای
gpt-5.6-terra	GPT 5.6 Terra	OpenAI	ChatGPT Pro	تحلیل چندمرحله‌ای و داده
glm-5.2	GLM 5.2	Z.AI ²	Other Pro Models	چندزبانه و کار سبک

Model ID	نام نمایشی	سازنده	گروه پیکر	نقش پیشنهادی
kimi-k3	Kimi K3	Moonshot AI	Other Pro Models	متون بلند و جمع‌بندی

۳.۲ نرمال‌سازی نام مدل (resolveModelId)

تابع `resolveModelId` در `src/lib/models.ts` (معادل `app.js:83`) ورودی را نرم تطبیق می‌دهد: ابتدا `trim` و `lowercase`، سپس فاصله‌ها و زیرخط‌ها به خط تیره (`/[\s_]+/ → "-"`)، بعد جست‌وجو در `id` ها و نام‌های نمایشی. نتیجه: «Claude Fable 5.1» و «claude_fable 5.1» و «claude-fable-5.1» هر سه به `claude-fable-5.1` می‌رسند. ورودی خالی ← مدل پیش‌فرض؛ ورودی ناشناخته دست‌نخورده عبور می‌کند – تصمیم عمدی: کلاینت‌هایی که نام مدل سفارشی می‌فرستند نباید 400 بگیرند، آپستریم خودش واکنش نشان می‌دهد.

۳.۳ افزودن مدل جدید (خلاصه Playbook)

مسیر کامل در بخش ۱۰ آمده؛ خلاصه پنج‌مرحله‌ای: رکورد `{id, name, vendor, group}` را در `models.ts` و دقیقاً همان را در `app.js` `FM_MODELS` اضافه کنید؛ لوگو را در `public/` بگذارید و با `scripts/embed_logos.py` در `EMBEDDED_LOGOS` (`app.js:79`) تزریق کنید؛ اگر گروه جدید است آیکن و برچسب گروه را در پیکر هر دو نسخه بسازید؛ با `GET /v1/models` و یک چت آزمایشی صحت‌سنجی کنید؛ و جدول مدل‌ها در این سند و README را به‌روز کنید – سندها جزو تعریف مدل‌اند، نه ضمیمه.

SECTION 04

استریم و پارسر جهانی SSE | Streaming & the Universal SSE Parser

۴.۱ چرا پارسر «جهانی»؟

آپستریم رایگان است و هیچ SLA فرمت ندارد؛ در عمل سبک غالبش OpenAI است اما بسته به مدل فیلدها متفاوت می‌آیند. به همین دلیل `createSseParser` در `src/lib/sse.ts` طراحی شده که «هر شکلی که آپستریم بگوید» را به یک قرارداد داخلی ساده تبدیل کند: رویداد `{text, reasoning, error?}`. همه مصرف‌کننده‌ها – UI چت، اندپوینت OpenAI، اندپوینت Anthropic – فقط این قرارداد را می‌بینند و از جزئیات آپستریم بی‌خبرند. اگر روزی آپستریم فرمتش را عوض کند، اولین و معمولاً تنها جای تغییر همین فایل است (معادل سرور در `app.js: makeUpstreamParser` خط ۲۳۹۹).

۴.۲ فرمت‌های پشتیبانی‌شده

الگو	نمونه / رفتار
سبک OpenAI (delta)	<code>choices[0].delta.{content, text}</code> + فیلدهای تفکر / <code>reasoning_content</code> <code>reasoning / thinking</code>
سبک OpenAI (غیراستریم)	<code>choices[0].message.{content, text, ...}</code> به همان قواعد
سبک Claude	<code>type: "content_block_delta"</code> با <code>delta.text</code> / <code>delta.thinking</code>
فیلد مستقیم ریشه	<code>{content}</code> یا <code>{text}</code> یا <code>{reasoning_content}</code>
آبجکت خام / رشته خام	بدون پیشوند <code>data:</code> هم پارس می‌شود؛ رشته غیر JSON مستقیم به متن چسبانده می‌شود
متادیتای SSE	[DONE] پایان استریم؛ <code>event:/id:/retry:</code> نادیده؛ خط : کامنت (مثل ping) رد می‌شود
خطای داخل استریم	آبجکت دارای <code>error</code> به‌صورت رویداد خطا منتشر می‌شود نه کرش

پارسر خطبه‌خط کار می‌کند و دو بافر دارد: `buf` برای خطوط ناقص (تکه‌های شبکه با مرز دلخواه می‌آیند) و `pendingJson` برای JSON چندخطی ناقص – تا وقتی `JSON.parse` موفق نشود `payload` جمع می‌شود. اگر در پایان استریم JSON ناتمامی بماند، به‌صورت خام منتشر می‌شود و هیچ داده‌ای دور ریخته نمی‌شود. این طراحی یعنی پارسر هرگز منتظر «چانک کامل» نمی‌ماند و تأخیر صفر دارد – شرط لازم برای کرسر زندهٔ AI.

دو ابزار سروری روی همین پارسر سوارند: `(app.js:2520 collectUpstreamText)` کل استریم را به متن کامل جمع می‌کند (برای پاسخ‌های غیراستریم /v1) و در نسخهٔ Next همین کار در مسیر غیراستریم با `(parser.feed(full) + parser.end())` انجام می‌شود. فیلد `reasoning` جدا از `text` منتشر می‌شود تا AI باکس «فراپند تفکر مدل» را بسازد و اندپوینت Anthropic بلوک `thinking` مستقل تولید کند.

SECTION 05

اندپوینت‌های سازگار OpenAI | OpenAI-Compatible API

۵.۱ احراز هویت و لیست مدل‌ها

```
Authorization: Bearer <key> # پذیرفته می‌شود (sk-... و sk-ant-...) هر دو کلید
# error.code = "invalid_api_key" کلید نامعتبر ← 401 یا

curl -s http://localhost:3000/v1/models \
  -H "Authorization: Bearer sk-ItqvFVBt8slw2vqxPuW9oLiSyGGgC29lNYeET64dIzJ04e50"
# owned_by = freemodels-anthropic|openai|z.ai|moonshot-ai ۷ مدل OpenAI لیست استاندارد ←
# بدون کلید ← 401
```

۵.۲ POST /v1/chat/completions

```
curl -sN http://localhost:3000/v1/chat/completions \
  -H "Authorization: Bearer sk-ItqvFVBt8slw2vqxPuW9oLiSyGGgC29lNYeET64dIzJ04e50" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "claude-fable-5.1",
    "messages": [{"role": "user", "content": "سلام! یک شعر کوتاه بگو"}],
    "stream": true,
    "thinking": false,           // اکستنشن غیراستاندارد (اختیاری)
    "deep_search": false        // اکستنشن غیراستاندارد (اختیاری)
  }'
```

قواعد ورودی: `messages` اجباری و باید غیرخالی باشد (400). نقش‌های `assistant` / `system` عیناً عبور می‌کنند و هر نقش ناشناخته به `user` نگاشت می‌شود (رفتار نرم). محتوای رشته‌ای یا آرایهٔ بلوکی هر دو پذیرفته است (`flattenContent`) بلوک‌ها را به متن ساده می‌کاهد – معادل `(app.js:2372 resolveModelId)`. نام مدل با `resolveModelId` نرمال می‌شود.

مسیر غیراستریم: پاسخ استاندارد `chat.completion` با `finish_reason:"stop"` و `usage` تخمینی – هر ۴ نویسه = ۱ توکن (آپستریم شمارندهٔ واقعی نمی‌دهد؛ `estTokens` در `v1.ts`). **مسیر استریم:** توالی `chat.completion.chunk` – چانک اول `{delta:{role:"assistant",content:""}}`؛ متن در `delta.content`؛ تفکر در `delta.reasoning_content` (همان فیلدی که SDK ها `reasoning` می‌شناسند)؛ چانک پایانی `finish_reason:"stop"`؛ با `stream_options.include_usage=true` یک چانک `usage` دار قبل از `data: [DONE]` می‌آید. قطع کلاینت وسط استریم ← آپستریم بلافاصله `destroy` می‌شود تا سهمیه هدر نرود.

```
# Python – SDK رسمی OpenAI
from openai import OpenAI
client = OpenAI(base_url="http://localhost:3000/v1",
                api_key="sk-ItqvFVBt8slw2vqxPuW9oLiSyGGgC29lNYeET64dIzJ04e50")
stream = client.chat.completions.create(
    model="claude-fable-5.1",
    messages=[{"role": "user", "content": "سلام!"}],
    stream=True,
)
for chunk in stream:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end="", flush=True)
```

SECTION 06

اندپوینت سازگار Anthropic | Anthropic-Compatible API

۶.۱ POST /v1/messages – قواعد ورودی

```
x-api-key: <key> # به عنوان جایگزین پذیرفته می شود Bearer هدر
anthropic-version: (اعتبارسنجی نمی شود) اختیاری است

curl -sN http://localhost:3000/v1/messages \
-H "x-api-key: sk-ant-api03-9lamB2K7KoYT8fw8tAp2CKL5kKs0HKhN4mE9eawuzRuW0aU3umJirsEFbS3B6RCjLRw8lrJ0YRmsnxECIu3nHaKM" \
-H "content-type: application/json" \
-d '{"model": "claude-fable-5.1", "max_tokens": 1024, "stream": true,
  "messages": [{"role": "user", "content": "خلاصه معماری پروژه را بده"}]}'
```

این اندپوینت سختگیرانه مطابق قوانین خود Anthropic است: `model` و `max_tokens` اجباری اند و نبودشان ← 400 با پیام عیناً SDK – `model: Field required` / `max_tokens: Field required` ها و ابزارهای رسمی به این خطاها تکیه دارند؛ این رفتار را شل نکنید (نکته بحرانی شماره ۱۰ در HANDOFF). فیلد `system` رشته ساده یا آرایه بلوک می پذیرد (به متن تبدیل و به عنوان پیام `system` به آپستریم می رود). نقش ها فقط `user/assistant` (باقیه ← `user`).

۶.۲ چرخه استریم و پاسخ کامل

چرخه استریم استاندارد Anthropic: (آبجکت `message` با `usage` ورودی) ← `ping` ← توالی بلوک ها: هر بلوک با `content_block_start` باز، با `content_block_delta` پر و با `content_block_stop` بسته می شود. تفکر مدل بلوک مستقل `thinking` با `thinking_delta` است و پاسخ نهایی بلوک `text` با `text_delta` – جابه جایی بین تفکر و پاسخ، بلوک قبلی را می بندد و بلوک جدید را با `index` بعدی باز می کند. پایان: `message_delta` (با `stop_reason: "end_turn"` و `message_stop` ← `output_tokens`). این ترتیب دقیقاً همان چیزی است که Anthropic SDK برای بازسازی استریم انتظار دارد.

غیراستریم: آبجکت `message` با آرایه `content` از بلوک ها – اگر تفکر موجود باشد `[{"type": "thinking", ...}]` و اگر نه فقط بلوک `text`. خطای 429 آپستریم این جا با `type: "rate_limit_error"` برگردانده می شود (نه `api_error`) تا SDK منطق `retry` خودش را اجرا کند.

```
# Python – SDK رسمی Anthropic
import anthropic
client = anthropic.Anthropic(
    base_url="http://localhost:3000",
    api_key="sk-ant-api03-9lamB2K7KoYT8fw8tAp2CKL5kKs0HKhN4mE9eawuzRuW0aU3umJirsEFbS3B6RCjLRw8lrJ0YRmsnxECIu3nHaKM",
)
with client.messages.stream(
    model="claude-fable-5.1",
    max_tokens=1024,
    messages=[{"role": "user", "content": "سلام!"}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

SECTION 07

قرارداد داخلی چت | Internal Chat Contract (/api)

UI چت (هر دو نسخه) با لایه AI از طریق این قرارداد ساده‌تر حرف می‌زند – این یک API عمومی نیست و برخلاف v1/ نیازی به کلید ندارد. تفاوت مهم `/api/chat` با `/v1/*` این است که این‌جا پروکسی خام هستیم: پاسخ آپستریم بدون تبدیل فرمت pipe می‌شود و پارس/نمایش در کلاینت انجام می‌گیرد (همان منطق پارسر SSE، سمت مرورگر). نام مدل این‌جا هم با `resolveModelId` نرمال می‌شود.

```
POST /api/chat {messages:[{role,content}], modelId, thinking, deepSearch, stream}
  ← کامل JSON یا (delta.content + delta.reasoning_content) آپستریم OpenAI سبک SSE استریم
  ← با 413/502/504 یا عبور کد آپستریم (مثلاً 429) {error:{message}}: خطا

GET /api/ping → {status:"ok"|"error", ms:number, sample:string(≤200)}
GET /api/keys → {openai, anthropic} # تزریق می‌کند window.__FM__ یا Next! فقط
```

برای دکمه «اتصال؟» است: یک درخواست سبک `stream:false` با پیام ping می‌فرستد و تأخیر واقعی آپستریم را گزارش می‌کند – پاسخ ۲ تا ۳ ثانیه‌ای طبیعی است و حتی وقتی آپستریم خطای سهمیه می‌دهد، endpoint خودش سالم ارزیابی می‌شود چون پیام آپستریم در sample دیده می‌شود.

SECTION 08

کلیدها و احراز هویت | API Keys & Authentication

کلیدها در `api-keys.json` (ریشه پروژه، 600 permission) نگهداری می‌شوند و بین هر دو نسخه مشترک‌اند:

```
{
  "openai": "sk-ItqvFVBt8slw2vqxPuW9oLiSyGGgC29lNYeET64dIzJ04e50",
  "anthropic": "sk-ant-api03-9lamB2K7KoYT8fw8tAp2CKL5kKs0HKhN4mE9eawuzRuW0aU3umJirsEFbS3B6RCjLRw8lrJ0YRmsnxECIu3nHaKM",
  "createdAt": "2026-09-14T..."
}
```

- قواعد تولید (اولین اجرا اگر فایل نباشد – `src/lib/apikeys.ts` و `loadOrCreateKeys()` در `app.js`): کلید سبک OpenAI = ۴۸ + `sk-` نویسه الفباعدی تصادفی؛ کلید سبک Anthropic = `sk-ant-api03-` + ۸۸ نویسه دقیقاً با الگوی رسمی تا اعتبارسنجی سمت کلاینت SDK ها رد نشود.
- اولویت: متغیرهای محیطی `OPENAI_API_KEY` / `ANTHROPIC_API_KEY` روی فایل `override` می‌کنند (برای استقرار بدون فایل).
- سیاست پذیرش: هر دو کلید روی هر دو اندپوینت کار می‌کنند – رفتار عمدی برای ساده نگه‌داشتن مصرف‌کننده.

- **کش و چرخش:** نتایج در حافظه کش می‌شود؛ برای اعمال تغییر فایل باید سرور را ری‌استارت کنید. چرخش کلید = ویرایش یا حذف فایل (اجرای بعدی می‌سازد) یا ست کردن env.

SECTION 09

مدیریت خطا و محدودیت‌ها | Error Handling & Limits

ماتریس کامل خطاها از دید مصرف‌کننده API – در همه مسیرها خطا «در فرمت مقصد» برمی‌گردد (آبجکت OpenAI یا ساختار `{type:"error",error:{type,message}}` آنتروپیک):

کد	معنا	ریشه	اقدام
400	invalid_request_error	JSON خراب، messages خالی، نبود model/max_tokens در v1/messages/	اصلاح درخواست
401	invalid_api_key	هدر احراز هویت غایب یا اشتباه	کلید صحیح (بخش ۸)
413	بدنه بیش از حد بزرگ	سقف ۵ = MAX_BODY_BYTES مگابایت	تاریخچه را کوتاه کنید
429	rate_limit_error (آنتروپیک)	سهمیه آپستریم «providers exhausted» – رفتار سرویس رایگان	چند ثانیه بعد retry – باگ نیست، مخفی هم نکنید
502	api_error	اتصال به آپستریم برقرار نشد (شبکه/قطعی)	ping بگیرید، بعداً تلاش کنید
504	api_error	تایم‌اوت ۱۸۰ ثانیه‌ای آپستریم	retry؛ اگر پایدار شد وضعیت آپستریم را ببینید

نکات ظریف برای توسعه‌دهنده: (۱) خطای 429 که «وسط» یک استریم باز برسد قابل تبدیل به کد HTTP نیست – در مسیر OpenAI به صورت `delta:{content:"⚠️ ..."}` و در مسیر Anthropic به صورت text_delta داخل بلوک text منتشر می‌شود تا کلاینت چیزی ببیند. (۲) پاسخ خالی آپستریم هرگز 200 خالی برنمی‌گردد؛ عبارت «⚠️ پاسخ خالی از سرور دریافت شد» جایگزین می‌شود تا SDK ها متن خالی را خطا نگیرند. (۳) CORS اندپوینت‌های v1/ باز است و هدرهای `x-api- Authorization`، `key` و `anthropic-version` را صریحاً مجاز می‌کند – از مرورگر هم مستقیم قابل مصرف است.

SECTION 10

کتاب راهنمای توسعه | Development Playbook

۱ افزودن فیلد اکستنشن جدید (مثل thinking / deep_search)

سه ایستگاه: (۱) در route مربوطه بخوانید و به payload آپستریم اضافه کنید (`parsed.my_field === true`)؛ (۲) همان فیلد را در payload builder داخل app.js اضافه کنید؛ (۳) اگر UI دارد، چک‌باکس را در هر دو فرانت اضافه کنید. نام فیلد در `v1/chat/completions/` از قاعده snake_case پیروی می‌کند (مثل deep_search) و در payload داخلی camelCase می‌شود (deepSearch) – این نداشت عمدی است تا با استاندارد OpenAI سازگار بمانیم.

۲ تغییر یا تعویض آپستریم

UPSTREAM_URL در `src/lib/upstream.ts` و ثابت هم‌نام در `app.js` (بالای openUpstream) را هم‌زمان عوض کنید؛ هدرهای SPOOFED_HEADERS را با دامنه جدید هماهنگ کنید؛ و اگر فرمت پاسخ عوض شد، طبق بخش ۴ اول `src/lib/sse.ts` و معادل `makeUpstreamParser` را اصلاح کنید – بقیه سیستم نباید متوجه شود.

```
K="sk-ItqvFVBt8slw2vqxPuW9oLiSyGGgC29lNYeET64dIzJ04e50"
A="sk-ant-api03-9lamB2K7KoYT8fw8tAp2CKL5Ks0HKHn4mE9eawuzRuW0aU3umJirsEFbS3B6RCjLRw8lrJ0YRmsnxECIu3nHaKM"
B=http://localhost:3000
curl -s $B/api/ping # سلامت آپستریم ۱
curl -s $B/v1/models -H "Authorization: Bearer $K" | head -c 400 # لیست ۷ مدل ۲
curl -s $B/v1/models | head -c 200 # بدون کلید 401 ۳
curl -sN $B/v1/chat/completions -H "Authorization: Bearer $K" \
-H "Content-Type: application/json" \
-d '{"model": "kimi-k3", "messages": [{"role": "user", "content": "سلام"}], "stream": true}'
# ۴ استریم + [DONE]
curl -s $B/v1/chat/completions -H "Authorization: Bearer $K" \
-H "Content-Type: application/json" \
-d '{"model": "glm-5.2", "messages": [{"role": "user", "content": "سلام"}]}'
# ۵ غیراستریم + usage
curl -s $B/v1/messages -H "x-api-key: $A" -H "Content-Type: application/json" \
-d '{"model": "claude-fable-5.1"}' # 400 ب ۶ max_tokens: Field required
```

۴ دیباگ و بهترین شیوه‌ها

لاگ سرور: هر درخواست با لاگ رنگی ANSI (زمان/مسیر/کد آپستریم/مدت) در کنسول هر دو نسخه چاپ می‌شود – اولین نقطه شروع دیباگ. **پنل Raw SSE:** در الی‌ا، دیتای خام آپستریم را تا ۸۰ کیلوبایت نشان می‌دهد؛ برای تشخیص «مشکل از فرمت آپستریم است یا از تبدیل ما» بی‌رقیب است. **نام مدل‌ها همیشه از منبع واحد** – هرگز id مدل را در فایل دیگری hardcode نکنید. **تفکر و deep_search هزینه‌دارند** (کندتر و پرخطاتر) – پیش‌فرض false را در نمونه‌ها ترویج کنید. **پس از هر تغییر:** bun run lint (صفر خطا) + node --check app.js + شش curl بالا.

SECTION 11

نقشه راه لایه AI | AI Layer Roadmap

اولویت‌بندی پیشنهادی بر اساس نسبت ارزش به هزینه: (۱) **Retry خودکار** با **backoff** برای 429 در openUpstream – یک retry با تأخیر ۲-۵ ثانیه بیشترین بهبود تجربه را با کمترین ریسک می‌دهد. (۲) **فیلدهای اختیاری group/vendor در پاسخ v1/models/** – سازگار با OpenAI می‌ماند چون فیلد اضافه است و کلاینت‌های بالادستی می‌توانند گروه‌بندی کنند. (۳) **شمارش توکن واقعی‌تر** برای usage (به‌جای نویسه تقسیم بر ۴). (۴) **Rate limit و audit log سمت خودمان** اگر پروژه عمومی شود. (۵) **پشتیبانی ورودی تصویر/فایل** در صورت باز شدن آن در آپستریم. هر آیتم باید طبق Playbook بخش ۱۰ در هر دو نسخه هم‌زمان پیاده شود، با شش curl تأیید گردد و ردیفی در `worklog.md` بگیرد تا زنجیره تاریخچه پروژه قطع نشود.

۱۱.۱ مرجع سریع فایل‌های لایه AI

نقش	نسخه Next.js	معادل در app.js
کاتالوگ مدل‌ها + نرمال‌سازی	src/lib/models.ts	resolveModelId · FM_MODELS (خط ۶۶) (۸۳)
انتقال آپستریم + هدرهای جعلی	src/lib/upstream.ts	openUpstream (۲۰۸۳)
پارسر جهانی SSE	src/lib/sse.ts	makeUpstreamParser (۲۳۹۹) · collectUpstreamText (۲۵۲۰)

نقش	نسخه Next.js	معادل در app.js
ابزار مشترک (/CORS، SSE، flatten، usage)	src/lib/v1.ts	توابع خطی کنار هاندلرها (flattenContent) (۲۳۷۲)
کلیدها	src/lib/apikeys.ts	loadOrCreateKeys + getBearerToken (۲۳۴۷)
اندپوینت OpenAI	src/app/v1/chat/completions/route.ts + v1/models/route.ts	هاندلرهای /v1/chat/completions و /v1/models
اندپوینت Anthropic	src/app/v1/messages/route.ts	هاندلر /v1/messages + anthropicError (۲۳۶۷)
پروکسی داخلی چت	src/app/api/chat/route.ts + api/ping + api/keys	هاندلرهای /api/chat و (+ /api/ping (window.__FM__

۱۱.۲ همبسته سندهای پروژه

سند	نقش	فرمت
README.md	داکیومنت کامل پروژه: معماری، نصب و اجرا، مرجع API مقدماتی، تاریخچه و نقشه راه کلی	MD + PDF
HANDOFF.md	سند تحویل: TL;DR سه دستوری، ۱۰ نکته بحرانی، قرارداد فرانت-بک، چکلیست صحت سنجی	MD + PDF
AI-GUIDE.md (همین سند)	مرجع لایه AI: مدل‌ها، آپستریم، SSE، اندپوینت‌های /v1، کلیدها، خطاها، Playbook	MD + PDF
worklog.md	گزارش خام تسک‌ها با جزئیات تست و تصمیم‌ها – منبع تاریخچه	MD

SMART CHAT – AI DEVELOPMENT GUIDE · V1.0 · SEPTEMBER 2026

این سند را به همراه **README.md** (داکیومنت کامل پروژه) و **HANDOFF.md** (سند تحویل و نکات بحرانی) بخوانید؛ مرجع اندپوینت‌های README مقدماتی‌تر و این سند عمیق‌تر و کاربردی‌تر است. پس از هر تغییر در لایه AI، بخش مربوطه همین راهنما و جدول مدل‌ها را همگام به‌روزرسانی کنید و ردیف جدیدی در **worklog.md** ثبت کنید.